INTERNSHIP PROJECT PRESENTATION

EmpFlow

A Cloud-Based Employee Database & Workforce Management System

Flutter | Firebase | Cloud Firestore | Material Design 3



Built end-to-end across mobile and web

What EmpFlow Is, and the Problem It Solves

Project Overview

EmpFlow is a cross-platform employee database and workforce management application built during my internship to give an organization's HR and admin team a single, secure place to manage employee records — replacing scattered spreadsheets with a real-time, role-governed cloud system.

Built for HR & Admin teams of an organization

Platforms Android, iOS, and Web from one codebase

Backend Firebase + Cloud Firestore (NoSQL)

THE PROBLEM STATEMENT

Most small and mid-sized teams still manage employee data through spreadsheets and email — with no central source of truth, no search, and no control over who can view or edit sensitive records.

- Records split across spreadsheets, emails, and local folders
- No centralized search — every lookup is manual
- No role-based access — all staff can view all records
- Onboarding and audits take far longer than they should

02 CURRENT STATE

Challenges With the Existing Manual System

Slow Employee Lookup

Finding one employee record in a spreadsheet of a few hundred rows took an HR staff member several minutes, multiplied across dozens of daily requests.

No Single Source of Truth

Copies of the same employee database existed in multiple spreadsheets and inboxes, frequently going out of sync with each other.

No Access Control

Any staff member with file access could view or edit every employee's salary and personal details — no separation between admin and general staff.

Manual Report Generation

Producing a PDF or Excel report for audits or salary processing meant manually copying and reformatting data each time.

EmpFlow: One Platform for All Employee Operations

A single Flutter + Firebase application that consolidates employee data, search, identification, exports, and alerts into one role-governed system.

Identity & Access

Firebase Auth with admin / staff role-based access control

Employee Database

Full create, view, update, and remove lifecycle for employee profiles

Smart Search

Multi-field, real-time filtering by name, employee ID, or department

QR Identification

Generate and scan QR codes for instant employee verification

Export Engine

One-tap PDF reports and Excel sheets for audits and salary processing

Notifications

Firebase push alerts with a local-notification fallback layer

Technology Stack

Frontend

- Flutter 3.x + Dart
- Material Design 3
- Provider state management

Auth & Security

- Firebase Authentication
- Role-based access control
- Shared Preferences sessions

Database

- Cloud Firestore (NoSQL)
- Real-time listeners
- Structured collections

Cloud Services

- Firebase Storage
- Firebase Cloud Messaging
- Firebase Hosting (Web)

Key Libraries

- qr_flutter, mobile_scanner
- pdf, excel_dart
- Lottie, Slidable

Tooling

- Git + GitHub
- VS Code / Android Studio
- Firebase CLI

Why Flutter over React Native: one codebase ships to Android, iOS, and Web with native performance and Material Design 3 out of the box.

Key Features

Employee CRUD

Structured onboarding, profile updates, and offboarding without IT intervention

Role-Based Access

Admin and staff views enforced at both the navigation and database rule level

Real-Time Search

Multi-field search by name, ID, or department with live, sub-second results

QR Identification

On-demand QR generation per employee, scanned via the device camera

PDF / Excel Export

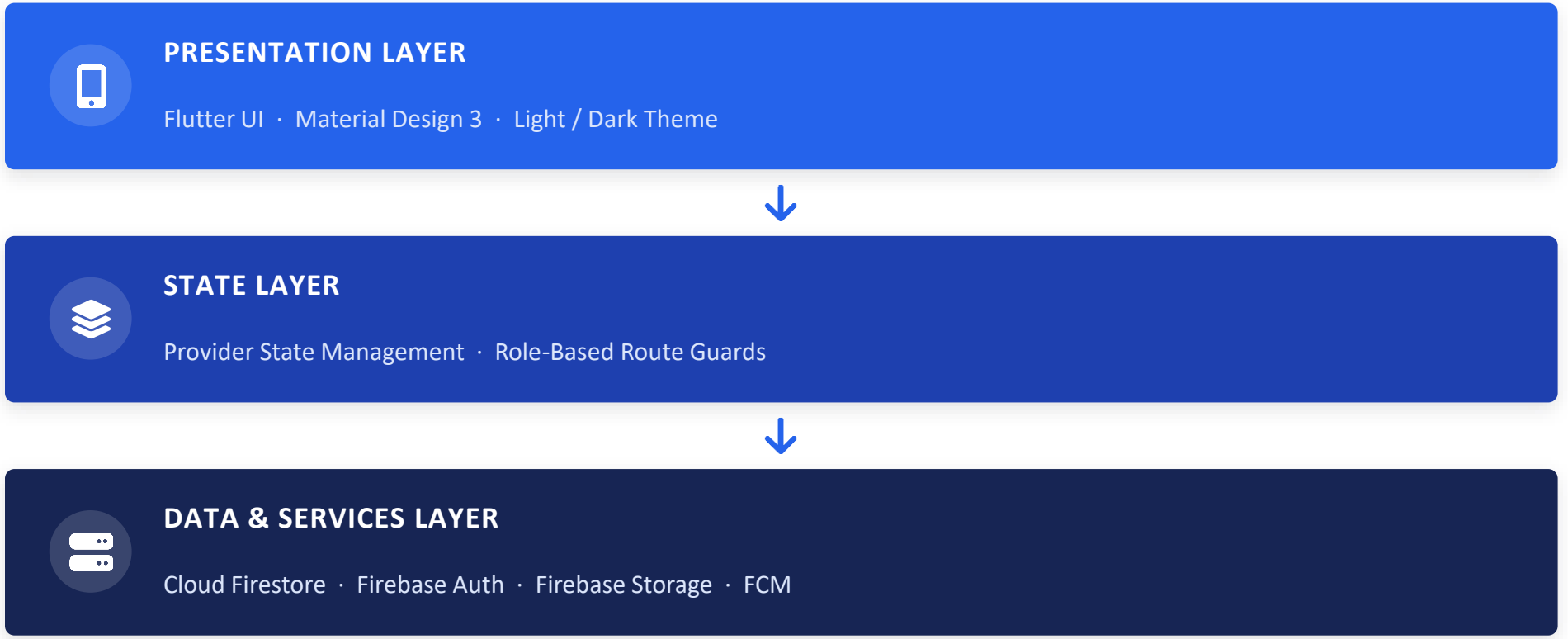
One-tap, audit-ready exports for HR reviews and salary processing

Push Notifications

FCM alerts for HR events with a local-notification fallback layer

System Architecture

Layered architecture with unidirectional build-up and real-time data flow back to the UI



Firestore listeners push real-time updates to Provider, which re-renders Flutter widgets automatically — no manual refresh needed.

Built Incrementally, Version by Version

v1.0 — Core Foundation

Employee CRUD, Firebase Auth, and the first Firestore schema

v1.1 — Faster Search

Indexed Firestore queries cut average search time well below 1 second

v1.2 — Optimized Firestore Reads

Restructured collections and added stream caching to reduce read calls

v1.3 — Stronger Authentication

Added role claims and route guards so admin and staff see different views

v1.4 — QR Identification

Introduced QR generation and camera-based scan-to-verify lookup

v1.5 — Export & Notifications

Added PDF / Excel export and FCM push alerts with local fallback

v2.0 — UI & Polish

Reworked UI with Material Design 3, dark theme, and smoother transitions

08 OUTCOMES

Results & Achievements

Measured during development and internal testing on a seeded organization dataset

~85%

faster employee lookup

vs. manual spreadsheet search, measured in internal testing

6

Firebase services integrated

Auth, Firestore, Storage, FCM, Hosting, and Cloud Messaging

150+

employee records tested

seeded dataset used to validate search, export, and QR flows

<1 sec

QR scan-to-profile time

from camera scan to full employee profile on screen

Why This Matters for the Organization

A self-built employee database of this kind removes dependency on paid third-party HRMS tools, which typically cost an organization roughly ₹15,000–₹25,000 per month for a similar feature set — while keeping full control over data and customization in-house.

09 WHAT'S NEXT

Future Scope

PHASE 2

Attendance Tracking

Geo-tagged check-in via QR or NFC, replacing manual registers

Leave Management

Self-service leave requests with multi-level approval workflows

PHASE 3

Analytics Dashboard

Real-time headcount, department distribution, and attrition trends

Salary Processing Integration

Connecting the employee database to a salary processing module

PHASE 4

Biometric Authentication

Fingerprint and face-ID login to remove password-based risk

In-App Communication

Contextual HR-to-employee messaging inside the platform

Learning Outcomes

Technical

- Built reactive UI with Provider + Firestore streams, no manual polling
- Designed a Firestore schema for efficient queries across collections
- Integrated 6 Firebase services into one coherent architecture

Product Thinking

- Mapped every feature back to a real business requirement first
- Iterated on UX through self-testing across 150+ sample records
- Learned to scope a roadmap and defer features to protect quality

Professional

- Version-controlled all changes via Git with structured commits
- Deployed to Firebase Hosting and understood the CI/CD lifecycle
- Documented architecture decisions, not just inline code comments

THANK YOU

EmpFlow · Flutter · Firebase · Cloud Firestore · FCM · QR Identification